Workforce Management System (WMS)

Project Technical Overview & System Architecture Specification

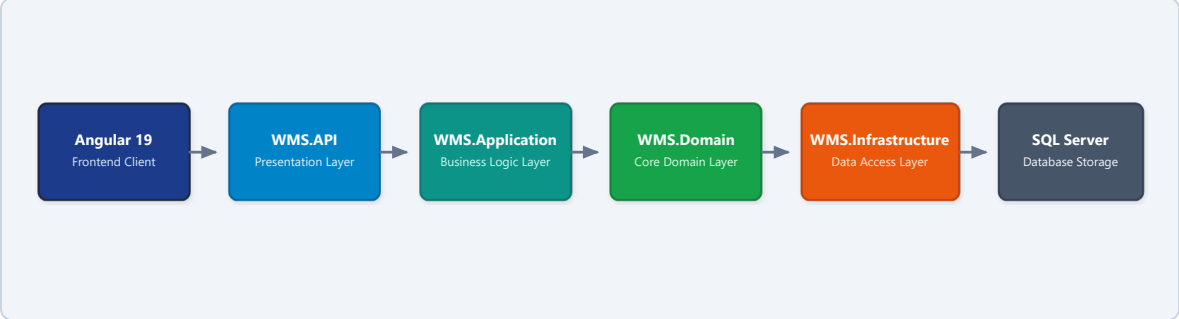
1. Introduction & Project Scope

The Workforce Management System (WMS) is an enterprise-grade web application built to automate and streamline core employee operations. The system is designed to handle employee records, department structures, daily attendance tracking, leave requests, and project resource allocations.

The project follows a modular, layered architecture to maintain a strict separation of concerns, ensuring high code maintainability, security, and testability.

2. System Architecture

The system is built as a multi-tier application connecting an Angular frontend, an ASP.NET Core Web API backend, and a SQL Server relational database store.

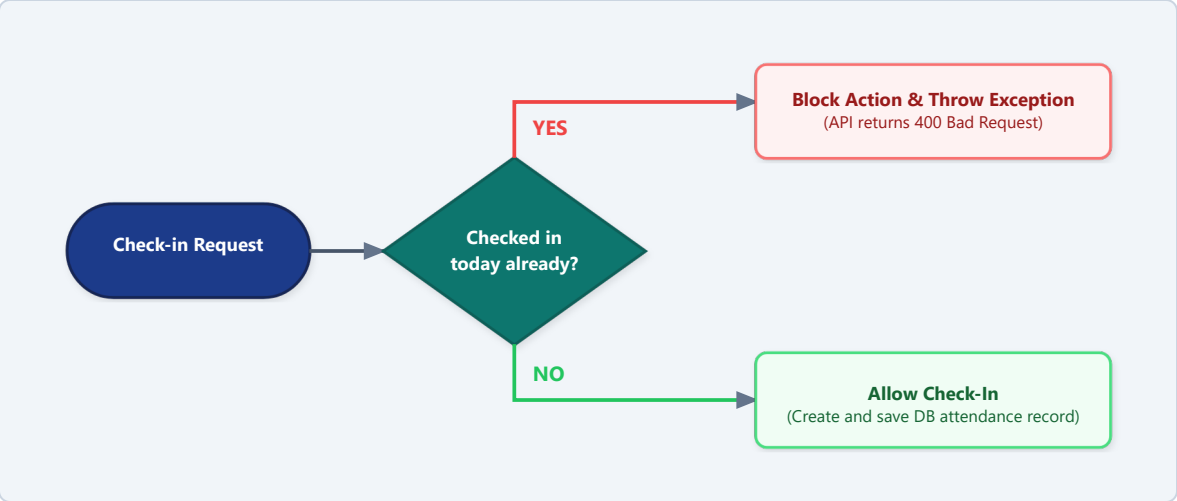


- **`WMS.Domain` (Core Domain)**: Defines fundamental models (Entities) and Interfaces. By design, this core has zero external project dependencies.
- **`WMS.Application` (Business Logic)**: Implements core services, handles DTO mappings (AutoMapper), triggers request validation (FluentValidation), and processes PDF logic.
- **`WMS.Infrastructure` (Data Access)**: Implements the repositories using EF Core Code-First, executing raw interactions against Azure SQL or local database servers.

3. Core Business Logic Workflows

Attendance Guard (Once-Per-Day Check-in Constraint)

A key operational rule enforced in the system prevents an employee from checking in multiple times in a single day. Once an employee has checked out for the day, they are blocked from starting another session until the next day.



Backend Guard (`WMS.Application`)

In `AttendanceService.cs`, when a check-in is initiated, the service queries the database for any records for that specific user occurring within the current calendar day range (`Today 00:00` to `Today 23:59`). If a record is found, the backend throws a validation exception which is caught globally and returned as a `400 Bad Request` API response.

Frontend Validation (`WMS.Frontend`)

The client application dynamically binds the "Check In" button's `disabled` attribute to the state variable `hasCheckedInToday`. Once checked out, this prevents a user from clicking the check-in button a second time in the UI.

4. Completed Requirements Mapping

This table demonstrates how features mapped from the project guidelines are implemented across the backend and frontend codebases:

Module	Requirements Coverage	Implementation Location
Employee Details	Add, edit, status toggle (Active/Inactive), search filters by name/department. Inactive employees locked out.	`EmployeesController`, `EmployeeService`
Attendance	Check-in and check-out logs, work mode metadata (WFO/WFH/Hybrid), PDF timesheet report export.	`AttendanceController`, `AttendanceService`
Leave Flows	Apply for Sick/Casual/Earned leave, track pending request tables, manager approval and rejection.	`LeavesController`, `LeaveService`
Projects & Clients	Client registration, Project CRUD, assign resources, manager-level approvals for allocations.	`ProjectsController`, `ProjectService`
Audit Logging	Database audit logs tracking user inserts, updates, and deletes for tracking and security compliance.	`AuditService`, `AuditLogTable`
Security & Auth	Token-based authorization using JSON Web Tokens (JWT), route protections using role guards in Angular.	`AuthService`, `RoleGuard`
CI/CD Pipelines	Integrated build and release pipelines compiled in YAML for automated tests, compiles, and Azure host pushes.	`azure-pipelines.yml`, `/WMS.DevOps`

> 💡 **Architectural Detail: Global exception handling is handled via custom ASP.NET Core middleware which captures exceptions and logs them properly while producing standardized error responses for the UI client.**

5. Environment Setup & Cloud Deployment

Project Prerequisite Configuration

- **SDK requirements**: .NET SDK 8.0 & Node.js LTS
- **Database**: SQL Server Express / Azure SQL / LocalDB

Live Deployment Endpoints on Microsoft Azure

Component	Azure Hosting Environment	Live Address
Angular Frontend	Azure Static Web Apps (SWA)	kind-ground-0fb549a00.7.azurestaticapps.net
Web API Backend	Azure App Service	wms-api-78807.azurewebsites.net/api
Swagger documentation	Swagger Open API Interface	wms-api-78807.azurewebsites.net/swagger/...
Database Store	Azure SQL Database Instances	`wmssqlserver78807.database.windows.net` (Database: `wmsdb`)

Pre-seeded Accounts for Verification:

- **Administrator Role**: Username: `admin` | Password: `Admin@123`
- **Manager Role**: Username: `manager` | Password: `Manager@123`
- **Employee Role**: Username: `employee` | Password: `Employee@123`

Running the Verification Tests:

The unit test suite verifies core repository interfaces and daily restriction guards in memory. You can run all test assertions locally by executing:

```
dotnet test
```

This command compiles and executes the test files within the `/WMS.Tests` directory and returns a full success log.
